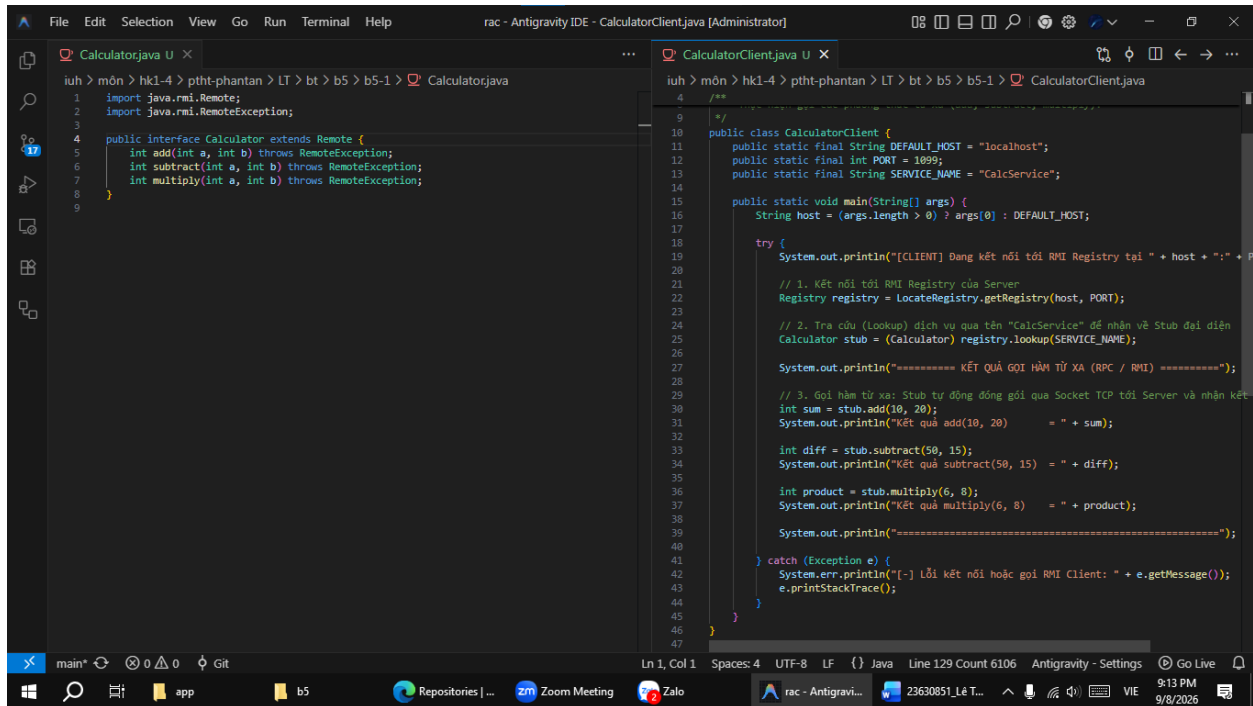


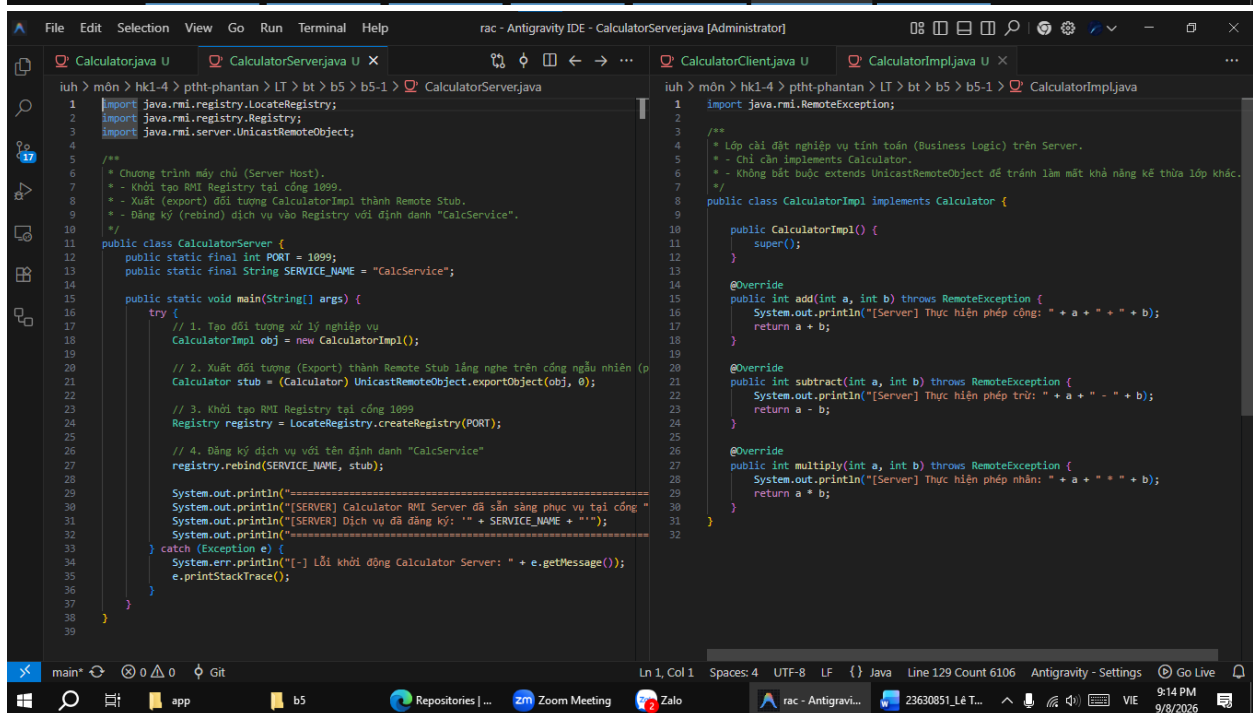
Lê Thành Long-23630851

B1



The screenshot shows the Antigravity IDE with the CalculatorClient.java file open. The code defines a Calculator interface and a CalculatorClient class. The interface has methods add, subtract, and multiply. The client class has a main method that connects to a remote registry, looks up the Calculator service, and performs calculations: 10 + 20, 50 - 15, and 6 * 8.

```
1 import java.rmi.Remote;
2 import java.rmi.RemoteException;
3
4 public interface Calculator extends Remote {
5     int add(int a, int b) throws RemoteException;
6     int subtract(int a, int b) throws RemoteException;
7     int multiply(int a, int b) throws RemoteException;
8 }
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
```



The screenshot shows the Antigravity IDE with two files open: CalculatorServer.java and CalculatorImpl.java. CalculatorServer.java implements the Calculator interface and registers the service. CalculatorImpl.java implements the Calculator interface and provides the business logic for the calculations.

```
1 import java.rmi.registry.LocateRegistry;
2 import java.rmi.registry.Registry;
3 import java.rmi.server.UnicastRemoteObject;
4
5 /**
6  * Chương trình máy chủ (Server Host).
7  * - Khởi tạo RMI Registry tại cổng 1099.
8  * - Xuất (export) đối tượng CalculatorImpl thành Remote Stub.
9  * - Đăng ký (rebind) dịch vụ vào Registry với định danh "CalcService".
10 */
11
12 public class CalculatorServer {
13     public static final int PORT = 1099;
14     public static final String SERVICE_NAME = "CalcService";
15
16     public static void main(String[] args) {
17         try {
18             // 1. Tạo đối tượng xử lý nghiệp vụ
19             CalculatorImpl obj = new CalculatorImpl();
20
21             // 2. Xuất đối tượng (Export) thành Remote Stub lắng nghe trên cổng ngẫu nhiên (p
22             Calculator stub = (Calculator) UnicastRemoteObject.exportObject(obj, 0);
23
24             // 3. Khởi tạo RMI Registry tại cổng 1099
25             Registry registry = LocateRegistry.createRegistry(PORT);
26
27             // 4. Đăng ký dịch vụ với tên định danh "CalcService"
28             registry.rebind(SERVICE_NAME, stub);
29
30             System.out.println("===== [SERVER] Calculator RMI Server đã sẵn sàng phục vụ tại cổng. =====");
31             System.out.println("[SERVER] Dịch vụ đã đăng ký: " + SERVICE_NAME + "");
32             System.out.println("=====");
33         } catch (Exception e) {
34             System.err.println("[!] Lỗi khởi động Calculator Server: " + e.getMessage());
35             e.printStackTrace();
36         }
37     }
38 }
39
40
41
42
43
44
45
46
47
```

B2

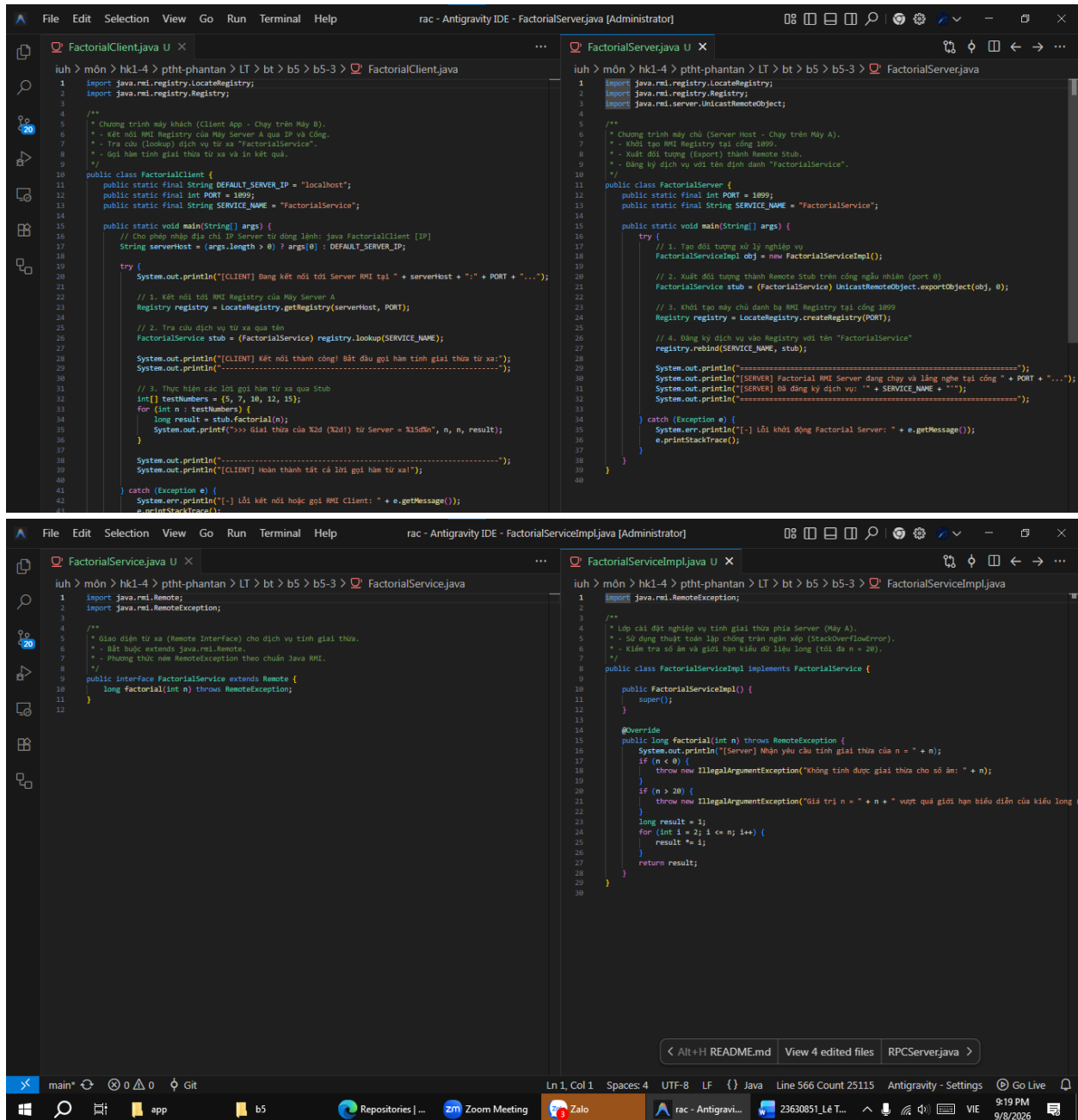
```
rac - Antigravity IDE - CalculatorClient.java [Administrator]
File Edit Selection View Go Run Terminal Help
Calculator.java U RPCServer.java U CalculatorClient.java U
iuh > môn > hk1-4 > ptht-phantan > LT > bt > b5 > b5-1 > RPCServer.java
1 /**
2  * Lớp RPCServer giả lập trong slide Bài 1.
3  * Đây là lớp cục bộ thông thường, chưa có cơ chế giao tiếp mạng hay RMI.
4  */
5 public class RPCServer {
6     public int add(int a, int b) {
7         System.out.println("[Server] Nhận yêu cầu cộng: " + a + " + " + b);
8         return a + b;
9     }
10 }
11

iuh > môn > hk1-4 > ptht-phantan > LT > bt > b5 > b5-2 > CalculatorClient.java
1 import java.rmi.registry.LocateRegistry;
2 import java.rmi.registry.Registry;
3
4 /**
5  * Chương trình máy khách (Client App - Bài 2).
6  * - Kết nối RMI Registry của Server.
7  * - Tra cứu (lookup) Stub đại diện cho "CalcService".
8  * - Gọi các hàm từ xa (add, subtract, multiply).
9  */
10 public class CalculatorClient {
11     public static final String DEFAULT_HOST = "localhost";
12     public static final int PORT = 1099;
13     public static final String SERVICE_NAME = "CalcService";
14
15     public static void main(String[] args) {
16         String host = (args.length > 0) ? args[0] : DEFAULT_HOST;
17
18         try {
19             System.out.println("[CLIENT] Đang kết nối tới RMI Registry tại " + host + ":" + PORT + "...");
20
21             // 1. Kết nối tới RMI Registry của Server
22             Registry registry = LocateRegistry.getRegistry(host, PORT);
23
24             // 2. Tra cứu (lookup) dịch vụ qua tên "CalcService" để nhận về Stub đại diện
25             Calculator stub = (Calculator) registry.lookup(SERVICE_NAME);
26
27             System.out.println("===== KẾT QUẢ GỌI HÀM TỪ XA (RPC / RMI) =====");
28
29             // 3. Gọi hàm từ xa: Stub tự động đóng gói qua Socket TCP tới Server và nhận kết quả
30             int sum = stub.add(10, 20);
31             System.out.println("Kết quả add(10, 20) = " + sum);
32
33             int diff = stub.subtract(50, 15);
34             System.out.println("Kết quả subtract(50, 15) = " + diff);
35
36             int product = stub.multiply(6, 8);
37             System.out.println("Kết quả multiply(6, 8) = " + product);
38             System.out.println("=====");
39
40         } catch (Exception e) {
41             System.err.println("[!] Lỗi kết nối hoặc gọi RMI Client: " + e.getMessage());
42             e.printStackTrace();
43         }
44     }
45 }
46
47
48 main main 0 0 Git Ln 1, Col 1 Spaces 4 UTF-8 LF { } Java Line 566 Count 25115 Antigravity - Settings Go Live
9:17 PM 9/8/2026
```

```
rac - Antigravity IDE - CalculatorImpl.java [Administrator]
File Edit Selection View Go Run Terminal Help
CalculatorImpl.java U README.md U RPCClient.java U CalculatorServer.java U
iuh > môn > hk1-4 > ptht-phantan > LT > bt > b5 > b5-2 > CalculatorImpl.java
1 import java.rmi.RemoteException;
2
3 /**
4  * Lớp cài đặt nghiệp vụ tính toán (Business Logic) trên Server.
5  * - Chỉ implements Calculator (không bị áp extends UnicastRemoteObject).
6  */
7 public class CalculatorImpl implements Calculator {
8
9     public CalculatorImpl() {
10         super();
11     }
12
13     @Override
14     public int add(int a, int b) throws RemoteException {
15         System.out.println("[Server] Thực hiện phép cộng: " + a + " + " + b);
16         return a + b;
17     }
18
19     @Override
20     public int subtract(int a, int b) throws RemoteException {
21         System.out.println("[Server] Thực hiện phép trừ: " + a + " - " + b);
22         return a - b;
23     }
24
25     @Override
26     public int multiply(int a, int b) throws RemoteException {
27         System.out.println("[Server] Thực hiện phép nhân: " + a + " * " + b);
28         return a * b;
29     }
30 }
31

iuh > môn > hk1-4 > ptht-phantan > LT > bt > b5 > b5-2 > CalculatorServer.java
1 import java.rmi.registry.LocateRegistry;
2 import java.rmi.registry.Registry;
3 import java.rmi.server.UnicastRemoteObject;
4
5 /**
6  * Chương trình máy chủ (Server Host - Bài 2).
7  * - Khởi tạo RMI Registry tại cổng 1099.
8  * - Xuất (export) đối tượng CalculatorImpl thành Remote Stub qua port ngẫu nhiên (port 0).
9  * - Đăng ký (rebind) dịch vụ vào Registry với tên định danh "CalcService".
10 */
11 public class CalculatorServer {
12     public static final int PORT = 1099;
13     public static final String SERVICE_NAME = "CalcService";
14
15     public static void main(String[] args) {
16         try {
17             // 1. Tạo đối tượng xử lý nghiệp vụ
18             CalculatorImpl obj = new CalculatorImpl();
19
20             // 2. Xuất đối tượng (Export) thành Remote Stub lắng nghe trên cổng ngẫu nhiên (port 0)
21             Calculator stub = (Calculator) UnicastRemoteObject.exportObject(obj, 0);
22
23             // 3. Khởi tạo RMI Registry tại cổng 1099
24             Registry registry = LocateRegistry.createRegistry(PORT);
25
26             // 4. Đăng ký dịch vụ với tên định danh "CalcService"
27             registry.rebind(SERVICE_NAME, stub);
28
29             System.out.println("=====");
30             System.out.println("[SERVER] Calculator RMI Server đã sẵn sàng phục vụ tại cổng " + PORT + "...");
31             System.out.println("[SERVER] Dịch vụ đã đăng ký: " + SERVICE_NAME + "...");
32             System.out.println("=====");
33         } catch (Exception e) {
34             System.err.println("[!] Lỗi khởi động Calculator Server: " + e.getMessage());
35             e.printStackTrace();
36         }
37     }
38 }
39
```

B3



BÀI 1: PHÂN TÍCH ĐOẠN CODE RPC GIẢ LẬP (CỘNG 2 SỐ)

Câu 1: Tại sao nói đoạn code trong slide CHƯA PHẢI là RPC thực sự?

- **Cùng không gian nhớ (Same Memory / JVM):** Client và Server chạy chung trong 1 tiến trình máy tính và 1 không gian RAM.
- **Là gọi hàm cục bộ (Local Call):** Lời gọi `server.add(5, 7)` thực hiện trực tiếp qua con trỏ bộ nhớ, hoàn toàn không có kết nối mạng (Socket TCP/IP) và không có cơ chế đóng gói dữ liệu (Marshalling).

Câu 2: RPC thật sự cần thêm những thành phần nào để chạy trên 2 máy khác nhau?

1. **Remote Interface (extends Remote):** Giao diện hợp đồng quy định các hàm dùng chung giữa 2 máy.
2. **Client Stub (Proxy):** Đại diện phía Client để **đóng gói (Marshalling)** tham số gửi qua mạng.
3. **Server Skeleton / Dispatcher:** Lắng nghe trên mạng, **giải nén (Unmarshalling)** và gọi hàm trên Server.
4. **Hạ tầng mạng (TCP Sockets):** Truyền tải các gói tin Request - Reply giữa 2 máy.
5. **Dịch vụ định danh (RMI Registry):** Danh bạ để Client tra cứu (Lookup) IP/Port của dịch vụ Server.

Câu 3: Muốn mở rộng nhiều hàm (add, subtract, multiply), ta cần thay đổi gì?

- **Interface:** Bổ sung khai báo chữ ký hàm mới (subtract, multiply).
- **Server:** Viết mã cài đặt logic tính toán cho các hàm đó.
- **Client:** Thực hiện gọi các hàm này thông qua đối tượng Stub.

BÀI 2: HỆ THỐNG RMI CALCULATOR CHUẨN 4 FILE

Câu 1: Tại sao Interface phải extends Remote và method phải throws RemoteException?

- **extends Remote:** Là **Marker Interface** đánh dấu cho JVM biết đối tượng này có thể được gọi từ xa qua mạng.
- **throws RemoteException:** Là **Checked Exception** bắt buộc Client phải có khối try-catch để xử lý các sự cố mạng tiềm ẩn (mất mạng, server sập, lỗi marshalling, timeout...).

Câu 2: Ý nghĩa của lệnh `UnicastRemoteObject.exportObject(obj, 0)` trong Server là gì?

- **Xuất Remote Stub:** Biến một đối tượng Java thông thường thành đối tượng từ xa có khả năng lắng nghe kết nối TCP/IP.
- **Ý nghĩa số 0:** Yêu cầu hệ điều hành tự động cấp phát một cổng ngẫu nhiên (Anonymous/Ephemeral Port) còn trống để tránh xung đột cổng.
- **Ưu điểm:** Giúp lớp `CalculatorImpl` không bị ép kế thừa `UnicastRemoteObject`, nhờ đó vẫn có thể kế thừa một lớp cha khác.

Câu 3: Nếu Server ở máy A (IP: 192.168.1.10) và Client ở máy B, thì Client cần sửa dòng nào?

- Sửa dòng kết nối Registry trong file CalculatorClient.java:

java

// Dòng cũ (chạy trên 1 máy):

```
Registry registry = LocateRegistry.getRegistry("localhost", 1099);
```

// Sửa lại (Client máy B trở sang Server máy A):

```
Registry registry = LocateRegistry.getRegistry("192.168.1.10", 1099);
```

Câu 4: So sánh điểm khác biệt chính giữa RPC và RMI trong Java:

Tiêu chí	RPC (Remote Procedure Call)	RMI (Remote Method Invocation)
Mô hình	Hướng thủ tục (Gọi Function / Procedure).	Hướng đối tượng (Gọi Method trên Remote Object).
Ngôn ngữ	Đa ngôn ngữ (C, C++, Python, Java...).	Chuyên biệt cho ngôn ngữ Java.
Truyền dữ liệu	Dữ liệu nguyên thủy hoặc cấu trúc phẳng (JSON, Struct).	Truyền được cả Đối tượng Java đầy đủ (qua Java Serialization).

Câu 5: Nếu bổ sung thêm hàm multiply(int a, int b), cần sửa những file nào?

Cần sửa **3 file**:

- Calculator.java: Thêm khai báo `int multiply(int a, int b) throws RemoteException;`
- CalculatorImpl.java: Cài đặt logic phép nhân `@Override public int multiply... { return a * b; }`.
- CalculatorClient.java: Bổ sung lời gọi hàm `stub.multiply(6, 8);`. (File *CalculatorServer.java* **không cần sửa**).

BÀI 3: DỊCH VỤ TÍNH GIAI THỪA PHÂN TÁN (FACTORIAL TRÊN 2 MÁY)

Câu 1: Nêu 2 điểm khác biệt chính giữa code giả lập và RPC thực sự:

1. **Tiến trình & Bộ nhớ:** Giả lập chạy chung 1 tiến trình và 1 bộ nhớ RAM; RPC chạy trên 2 tiến trình độc lập ở 2 máy tính riêng biệt qua mạng.
2. **Cơ chế truyền thông:** Giả lập gọi hàm qua con trỏ bộ nhớ CPU Stack; RPC phải đóng gói dữ liệu (**Marshalling**) thành luồng byte truyền qua mạng **TCP/IP**.

Câu 2: Muốn chạy RPC trên 2 máy khác nhau, ta cần bổ sung thành phần gì?

1. Tạo **Remote Interface** (FactorialService).
2. Cơ chế **Stub/Skeleton** (dùng exportObject).
3. Khởi chạy **RMI Registry** trên cổng 1099 tại máy Server.
4. Cấu hình **IP thực tế của Server** tại Client và mở cổng tường lửa (Firewall 1099) trên Server.

Câu 3: Nếu input \$n\$ rất lớn (ví dụ \$n = 50.000\$), gặp 4 vấn đề gì khi dùng RPC?

1. **Tràn số (Arithmetic Overflow):** Kiểu long chỉ chứa tối đa đến \$2^{31}-1\$. Với \$n = 50.000\$ kết quả có hơn 200.000 chữ số, phải dùng kiểu **BigInteger**.
2. **Tràn ngăn xếp (StackOverflowError):** Đề quy tạo ra 50.000 frame trên RAM làm sập Server, phải đổi sang **vòng lặp (for/while)**.
3. **Treo Client / Timeout mạng:** Tính toán số quá lớn tốn thời gian khiến Client bị treo chờ kết quả (Blocking) dẫn đến rớt mạng do timeout, cần đổi sang cơ chế **gọi bất đồng bộ (Asynchronous RPC / Future)**.
4. **Tải dữ liệu truyền mạng (Payload Overhead):** Kết quả hàng trăm KB làm tăng độ trễ và tốn băng thông truyền tải.

